

ViAmpleHate: Một hướng tiếp cận dựa trên AmpleHate và PhoBERT cho bài toán phát hiện phát ngôn thù ghét tiếng Việt

Tran Nguyen Duc Trung Nguyen Ha Minh Tuan Nguyen Gia Cat Long
Tran Phan Thanh Tung Mo Van Tung

Trường Đại học Công nghệ Thông tin, ĐHQG-HCM, Thành phố Hồ Chí Minh, Việt Nam
{23521687, 23521718, 23520881, 23521747, 23521741}@gm.uit.edu.vn

Abstract

Phát hiện phát ngôn thù ghét trên mạng xã hội tiếng Việt là một bài toán khó, vì ý đồ thù ghét không phải lúc nào cũng xuất hiện qua thực thể có tên mà thường ẩn trong cách gọi nhóm phi chính thức, tiếng lóng, mỉa mai và các ám chỉ gián tiếp. Các mô hình *target-aware* như AmpleHate cho thấy việc đưa quan hệ giữa phát ngôn và đối tượng bị nhắm tới vào cơ chế attention có thể hỗ trợ phát hiện cả những trường hợp thù ghét hàm ẩn. Tuy nhiên, pipeline gốc của AmpleHate phụ thuộc vào NER tiếng Anh và các phạm trù target được thiết kế cho tiếng Anh, nên khi chuyển sang tiếng Việt, hiệu quả suy giảm đáng kể: trên tập huấn luyện của nhóm, NER tiếng Anh chỉ tìm được target khả dụng trong khoảng 0,09% bình luận, khiến mô hình gần như thoái hóa thành một classifier ở mức câu. Nhóm đề xuất **ViAmpleHate**, một phiên bản thích ứng của AmpleHate cho tiếng Việt trên nền PhoBERT, với bốn thay đổi chính: (i) thay NER tiếng Anh bằng NER tiếng Việt kết hợp một *target-cue bank* được tuyển chọn, qua đó nâng độ phủ target lên khoảng 19–45% tùy bộ dữ liệu; (ii) bổ sung một *attack-cue bank* riêng và mô hình hóa ba kênh quan hệ—target tường minh, context hàm ẩn và attack—bằng *relation-bank attention*; (iii) thay phép injection cố định bằng *instance-adaptive gate*; và (iv) huấn luyện bằng weighted cross-entropy kết hợp contrastive loss. Trên ViHSD và VOZ-HSD, trong thiết lập nhị phân NON-HATE/HATE, ViAmpleHate cải thiện cả macro-F1 lẫn HATE-F1 so với baseline AmpleHate cũng như các baseline TF-IDF, BiLSTM và PhoBERT-CNN; mức tăng rõ nhất nằm ở lớp thiểu số (HATE-F1 tăng 0,0508 trên VOZ-HSD). Riêng trên ViHSD, mức cải thiện còn nhỏ, nên nhóm xem đây là kết quả sơ bộ và cần kiểm chứng thêm bằng đánh giá đa seed có kiểm soát. Phân

tích bổ sung cho thấy độ phủ target-cue có liên hệ với mức cải thiện; kết quả này ủng hộ nhận định rằng mô hình target-aware cần được thích ứng với tiếng Việt thay vì chuyển nguyên xi từ tiếng Anh, dù chưa đủ để khẳng định quan hệ nhân quả.

1 Giới thiệu

Sự phát triển nhanh của mạng xã hội tiếng Việt đi kèm với sự lan rộng của nội dung thù ghét và lăng mạ. Những nội dung này gây hại cho cá nhân, cộng đồng bị nhắm tới, đồng thời tạo rủi ro pháp lý và danh tiếng cho các nền tảng. Khi lượng nội dung vượt xa khả năng kiểm duyệt thủ công, phát hiện phát ngôn thù ghét tự động trở thành nhu cầu cấp thiết. Tuy nhiên, phần lớn tiến bộ hiện nay vẫn tập trung ở tiếng Anh và các ngôn ngữ giàu tài nguyên; khi áp dụng sang tiếng Việt, hiệu quả thường giảm đáng kể.

Phát hiện phát ngôn thù ghét tiếng Việt khó vì ba lý do đan xen. Thứ nhất, ngôn ngữ trên mạng xã hội rất nhiều: người dùng viết tắt, dùng *teencode*, kéo dài ký tự, chèn emoji mang sắc thái ngữ dụng và viết sai chính tả, đôi khi còn cố tình che lời tục để né bộ lọc. Thứ hai, và cũng là điểm quan trọng nhất, *target* của một lời công kích hiếm khi là một thực thể có tên. Nếu trong tiếng Anh, phát ngôn thù ghét thường gọi đích danh một cá nhân, tổ chức hoặc quốc tịch, thì trong tiếng Việt sự thù địch thường hướng tới cả một nhóm thông qua đại từ, danh từ thân tộc hay danh từ chỉ nhóm, nhân vùng miền hoặc các lối xưng hô phi chính thức. Thứ ba, dữ liệu bị mất cân bằng mạnh: ở cả hai bộ dữ liệu, lớp HATE chỉ chiếm khoảng một phần mười, nên mô hình vẫn có thể đạt accuracy cao dù gần như không phát hiện được trường hợp thù ghét nào.

Một hướng tiếp cận hứa hẹn là mô hình

hóa phát ngôn thù ghét theo kiểu *target-aware*. AmpleHate (Lee et al., 2025) cho thấy attention tường minh vào quan hệ giữa câu và target tiềm năng giúp phát hiện tốt hơn thù ghét *hàm ẩn*, tức những trường hợp không có slur lộ liễu. Cụ thể, mô hình trích các target ứng viên bằng NER, tính một attention vector target-aware rồi inject vector này vào biểu diễn câu trước khi phân loại. Hướng này phù hợp với tiếng Việt, nơi target thường giữ vai trò trung tâm, nhưng pipeline gốc lại dựa trên NER và phạm trù target của tiếng Anh. Khi chạy trên tiếng Việt với NER tiếng Anh, hệ thống chỉ tìm được target khả dụng ở 21/24.048 bình luận huấn luyện (khoảng 0,09%); vì vậy, phần lớn đầu vào phải quay về biểu diễn toàn cục [CLS], khiến mô hình gần như hoạt động như một PhoBERT classifier thông thường và đánh mất tín hiệu target-aware vốn là động lực chính.

Nhóm đề xuất **ViAmpleHate**, một phiên bản thích ứng của AmpleHate cho tiếng Việt, trong đó từng khâu của pipeline được điều chỉnh để phù hợp hơn với cách người Việt biểu đạt sự thù ghét trên mạng xã hội. Các đóng góp chính gồm:

1. **Trích xuất target tiếng Việt.** Nhóm thay NER tiếng Anh bằng một mô hình NER tiếng Việt kết hợp với *target-cue bank* được tuyển chọn (gồm đại từ, danh từ chỉ nhóm và các lỗi xưng hô phi chính thức), qua đó nâng tỷ lệ bình luận phát hiện được target từ khoảng 0,09% lên 18,8–20,0%.
2. **Tách riêng kênh target và kênh attack.** Nhóm bổ sung một *attack-cue bank* cho các vị từ thù địch, đồng thời xử lý cue target và cue attack như hai tín hiệu độc lập trong một **relation-bank attention** ba kênh.
3. **Fusion thích ứng (kèm ghi chú triển khai).** Nhóm thay phép injection cố định theo hệ số vô hướng của AmpleHate bằng **instance-adaptive gate**; ngoài ra, bài báo ghi nhận một chỉnh sửa ở mức triển khai cho phép tính attention theo batch trong bản port (chỉ xem là ghi chú triển khai, không phải đóng góp chính).
4. **Hàm mục tiêu huấn luyện.** Nhóm kết hợp weighted cross-entropy với contrastive loss để làm rõ hơn ranh giới giữa HATE

và NON-HATE trong điều kiện mất cân bằng.

5. **Nghiên cứu thực nghiệm.** Nhóm đánh giá trên ViHSD và VOZ-HSD với năm baseline, đồng thời phân tích nguồn gốc của các mức cải thiện, bao gồm độ phủ target và khảo sát định tính những lỗi còn lại.

target-aware là một inductive bias hữu ích cho bài toán phát hiện phát ngôn thù ghét, nhưng chỉ phát huy đầy đủ khi được thích ứng với đặc trưng bề mặt của ngôn ngữ đích, thay vì chuyển nguyên xi từ tiếng Anh.

2 Công trình liên quan

2.1 Phát hiện phát ngôn thù ghét và thù ghét hàm ẩn

Các phương pháp phát hiện phát ngôn thù ghét tự động đã phát triển từ classifier dựa trên từ điển và đặc trưng thủ công, qua các mô hình chuỗi sâu, đến các transformer tiền huấn luyện gần đây. Một trở ngại dai dẳng là thù ghét *hàm ẩn*: thông điệp thể hiện sự thù địch mà không dùng slur tường minh, thay vào đó dựa vào mỉa mai, định kiến, tham chiếu được mã hóa hoặc kiến thức nội nhóm. Dạng này thường gặp hơn slur lộ liễu và đặc biệt khó đối với các hệ thống dựa trên từ khóa, qua đó thúc đẩy những hướng tiếp cận có khả năng suy luận *ai bị nhắm tới và bị nhắm tới theo cách nào*.

2.2 AmpleHate

AmpleHate (Lee et al., 2025) xử lý thù ghét hàm ẩn bằng cách khuếch đại attention giữa biểu diễn toàn cục của câu và các target ứng viên. Mô hình trích target token bằng NER, tính một HeadAttention vector với query từ [CLS] và key/value từ target token, rồi inject kết quả này với một hệ số cố định trước khi phân loại. Quan điểm cốt lõi của mô hình, rằng target là tín hiệu then chốt, trực tiếp truyền cảm hứng cho công trình này. Tuy vậy, các hạn chế khi áp dụng cho tiếng Việt cũng khá rõ: mô hình giả định NER và phạm trù target kiểu tiếng Anh, inject thông tin quan hệ với cùng cường độ cho mọi mẫu, và chỉ mô hình hóa một quan hệ (target) mà bỏ qua vị từ thù địch (*attack*), tức yếu tố biến một lời nhắc tên thành lời thù ghét.

2.3 Phát hiện phát ngôn thù ghét tiếng Việt

Tiếng Việt ngày càng có nhiều tài nguyên và mô hình cho bài toán phát hiện phát ngôn thù ghét. ViHSD (Luu et al., 2021) cung cấp một bộ dữ liệu ba lớp (CLEAN/OFFENSIVE/HATE) quy mô lớn và đã trở thành một benchmark tiêu chuẩn. ViTHSD (Vo et al., 2025) đặt lại bài toán xoay quanh *target* bằng cách chú thích mức độ thù ghét nhắm vào từng *target*; điều này cũng cố vai trò trung tâm của *target* và ủng hộ thiết kế *target-aware* của nhóm. Khác với ViTHSD, ViAmpleHate không đòi hỏi chú thích *target* ở mức span: mô hình tự định vị *target* và *attack* bằng NER kết hợp *cue bank*, nên vẫn áp dụng được cho các bộ dữ liệu chỉ có nhãn ở mức câu.

2.4 PhoBERT và các mô hình tiền huấn luyện tiếng Việt

PhoBERT (Nguyen and Nguyen, 2020) là mô hình kiểu RoBERTa được tiền huấn luyện trên kho ngữ liệu tiếng Việt lớn, hiện thường được dùng làm encoder mặc định cho phân loại văn bản tiếng Việt. Vì PhoBERT được huấn luyện trên văn bản đã được *tách từ* (word segmentation), bước tách từ cần được thực hiện trước khi token hóa. Các kiến trúc lai như PhoBERT-CNN gắn thêm bộ trích đặc trưng cục bộ lên trên embedding ngữ cảnh. Nhóm dùng PhoBERT-base làm encoder chung cho cả baseline lẫn mô hình đề xuất, để khác biệt về kết quả chủ yếu phản ánh cách mô hình hóa *target-aware* thay vì khác biệt ở backbone.

2.5 Contrastive learning cho phân loại văn bản

Các contrastive objective khuyến khích biểu diễn của các mẫu cùng lớp xích lại gần nhau và biểu diễn của các mẫu khác lớp tách xa ra, nhờ đó cải thiện độ ổn định và mức phân tách giữa các lớp, đặc biệt trong bối cảnh dữ liệu mất cân bằng (Khosla et al., 2020). Nhóm thêm một contrastive objective phụ trợ trên biểu diễn sau gate (post-gate), bên cạnh weighted cross-entropy. Đây là một số hạng cosine-margin theo cặp, lấy cảm hứng từ supervised contrastive learning chứ không phải SupCon loss nguyên bản, nhằm siết chặt ranh giới giữa lớp NON-HATE phổ biến và lớp HATE hiếm gặp.

3 Bộ dữ liệu

3.1 ViHSD

ViHSD (Luu et al., 2021) là bộ dữ liệu phát ngôn thù ghét tiếng Việt gồm các bình luận mạng xã hội, được chú thích theo ba nhãn gốc (CLEAN, OFFENSIVE, HATE) và đã chia sẵn thành train/validation/test. Sau bước gán nhãn nhị phân (Mục 3.3), phân phối của từng tập được trình bày trong Bảng 1.

3.2 VOZ-HSD

VOZ-HSD (Thanh Nguyen, 2024b), được phát hành cùng ViHateT5 (Thanh Nguyen, 2024a), gồm các bình luận từ diễn đàn VOZ (lấy từ bản công khai tarudesu/VOZ-HSD). Tương tự ViHSD, dữ liệu được chia thành train/validation/test, và phân phối sau khi gán nhãn lại được trình bày trong Bảng 1. Vì đây là tài nguyên của bên thứ ba, quy trình chú thích chi tiết và giấy phép dữ liệu do nhóm tác giả gốc công bố.

3.3 Gán nhãn nhị phân và tiền xử lý

Thao tác duy nhất nhóm thực hiện là *gán nhãn lại*; số lượng mẫu không bị thay đổi hay lấy mẫu lại. Cụ thể, bài toán được đưa về phân loại nhị phân bằng cách gộp CLEAN và OFFENSIVE thành NON-HATE, đồng thời giữ HATE làm lớp dương. Cách làm này giúp tập trung vào thù ghét hướng nhóm (thay vì lời tục hoặc sự gây hấn nói chung) và tạo ra một không gian nhãn nhất quán giữa các bộ dữ liệu. Mỗi bình luận sau đó được chuẩn hóa (viết thường; bỏ URL và ký hiệu; gộp các ký tự bị kéo dài; chuẩn hóa teencode; ánh xạ các emoji thường gặp về những nhãn ngữ dụng thô như chế giễu, tức giận, ghê tởm hoặc cười cợt), rồi tách từ—bước bắt buộc vì PhoBERT được tiền huấn luyện trên văn bản tiếng Việt đã được tách từ—và cuối cùng token hóa bằng tokenizer của PhoBERT.

3.4 Xây dựng *cue bank*

Hai thành phần trung tâm của ViAmpleHate là *target-cue bank* và *attack-cue bank*. Cả hai đều được nhóm xây dựng thủ công từ việc khảo sát tập train, kết hợp với các biểu thức tham chiếu và lăng mạ phổ biến trong tiếng Việt, rồi chuẩn hóa để khớp với pipeline tiền xử lý. **Target-cue bank** gồm những biểu thức thường dùng để chỉ một người hoặc một nhóm—đại từ, danh

Bộ dữ liệu	Tập	NON-HATE	HATE	Tổng
ViHSD	Train	21.492	2.556	24.048
ViHSD	Dev	2.402	270	2.672
ViHSD	Test	5.992	688	6.680
VOZ	Train	26.993	3.007	30.000
VOZ	Dev	4.520	480	5.000
VOZ	Test	4.487	513	5.000

Table 1: Thống kê bộ dữ liệu sau khi gán nhãn nhị phân. Lớp HATE chiếm $\approx 10\%$ ở mọi tập, nên macro-F1 và HATE-F1 phản ánh hiệu năng phù hợp hơn accuracy.

từ thân tộc hoặc danh từ chỉ nhóm, nhân vùng miền và nhân khẩu, cùng các lỗi xưng hô phi chính thức—nhưng bản thân chúng *không* phải dấu hiệu thù ghét. **Attack-cue bank** gồm các vị từ thù địch như lăng mạ, phi nhân tính hóa, đe dọa và đánh giá tiêu cực gay gắt. Việc tách hai bank là có chủ đích: một target không đi kèm vị từ thù địch thường chưa phải thù ghét, trong khi một vị từ thù địch không hướng tới target nhóm rõ ràng thường chỉ là xúc phạm. Mỗi cue được chuẩn hóa, tách từ, token hóa rồi đối sánh bằng khớp trọn chuỗi token, nhờ đó các cụm nhiều token được nhận diện như một đơn vị thay vì khớp nhầm vào subtoken rồi rạc. Hai bank được giữ nhỏ và tuyển chọn thủ công—16 target cue và 24 attack cue (Phụ lục B). Vì được khởi tạo từ dữ liệu train, chúng có nguy cơ mã hóa các mẫu đặc thù của tập này và bị giới hạn recall trước tiếng lóng mới; độ phủ được báo cáo ở Mục 6.2.

4 Phương pháp

4.1 Phát biểu bài toán

Với một bình luận tiếng Việt x , mô hình dự đoán $y \in \{\text{NON-HATE}, \text{HATE}\}$. ViAmpleHate kế thừa ý tưởng target-aware của AmpleHate và thích ứng ý tưởng này qua năm thay đổi: trích xuất target tiếng Việt, tách riêng hai kênh target/attack cue, relation-bank attention, phép tính attention theo batch đã được chỉnh sửa, và injection thông qua adaptive gate (Hình 1).

4.2 Tiền xử lý văn bản

Mỗi bình luận được chuẩn hóa, tách từ, token hóa bằng tokenizer của PhoBERT, rồi cắt hoặc đệm về độ dài `max_len = 256`.

4.3 Trích xuất đa tín hiệu target và attack

Tập chỉ số target được hợp nhất từ NER tiếng Việt và kết quả khớp target cue, còn tập chỉ số attack đến từ kết quả khớp attack cue. Nếu một trong hai tập rỗng, mô hình fallback về [CLS] để luôn có sẵn một biểu diễn hàm ẩn ở mức câu:

$$T_x = M_{\text{NER}}(x) \cup M_{\text{target}}(x), \quad A_x = M_{\text{attack}}(x) \quad (1)$$

$$T_x \leftarrow \{0\} \text{ nếu } T_x = \emptyset; \quad A_x \leftarrow \{0\} \text{ nếu } A_x = \emptyset$$

4.4 PhoBERT encoder

$$H = \text{PhoBERT}(x) = [h_0, h_1, \dots, h_n], \quad (2)$$

với mỗi $h_i \in \mathbb{R}^d$ và $d = 768$; trong đó h_0 là biểu diễn [CLS] (context toàn cục), còn các vị trí target/attack cung cấp bằng chứng cục bộ.

4.5 Relation-bank attention

Nhóm xây dựng ba góc nhìn quan hệ cho mỗi mẫu—target tường minh, context hàm ẩn và attack:

$$\begin{aligned} r_{\text{exp}} &= \text{HeadAttn}(h_0, H[T_x]), \\ r_{\text{imp}} &= \text{HeadAttn}(h_0, h_0), \\ r_{\text{atk}} &= \text{HeadAttn}(h_0, H[A_x]), \end{aligned} \quad (3)$$

trong đó mỗi module áp dụng trên $E \in \mathbb{R}^{m \times d}$ và tính

$$\alpha = \text{softmax}\left(\frac{(W_q h_0)(W_k E)^\top}{\sqrt{d}}\right), \quad r = \alpha (W_v E). \quad (4)$$

Cuối cùng, ba vector được fuse thành $r = W_r[r_{\text{exp}}; r_{\text{imp}}; r_{\text{atk}}] + b_r$.

4.6 Batched attention (ghi chú triển khai)

Trong quá trình tái triển khai, nhóm nhận thấy attention kiểu AmpleHate, nếu được tính bằng $QK^\top$ trên toàn bộ batch ($Q, K \in \mathbb{R}^{B \times d} \Rightarrow QK^\top \in \mathbb{R}^{B \times B}$), có thể trộn lẫn thông tin giữa các mẫu. Vì vậy, nhóm chuyển sang batched attention để mỗi mẫu chỉ attend tới các token của chính nó, với $Q \in \mathbb{R}^{B \times 1 \times d}$, $K \in \mathbb{R}^{B \times m \times d}$:

$$\text{scores} = \text{bmm}(Q, K^\top) / \sqrt{d} \in \mathbb{R}^{B \times 1 \times m}, \quad (5)$$

trong đó phần padding được mask trước softmax ($\text{scores}_j = -\infty$ nếu $\text{mask}_j = 0$). Nhóm xem đây là một chỉnh sửa ở mức triển khai cho bản port của mình, không phải một khẳng

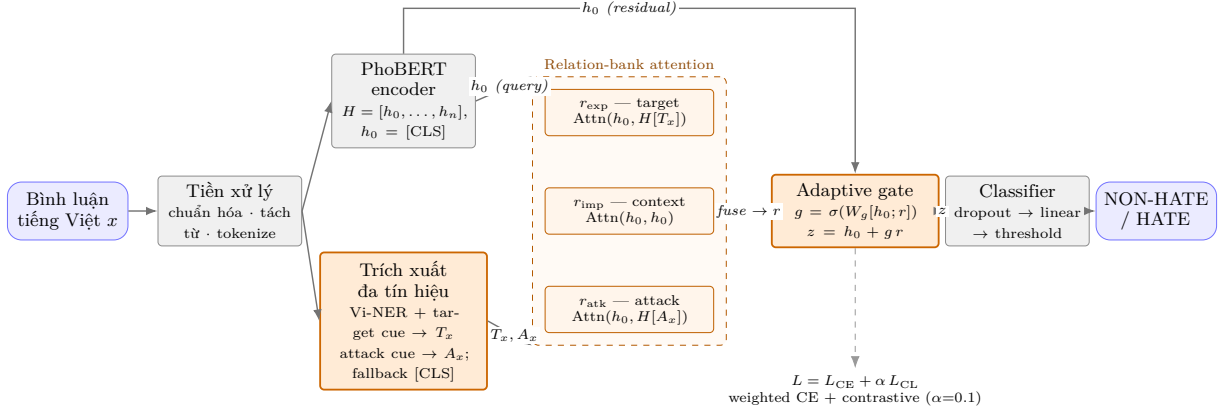


Figure 1: Kiến trúc tổng thể của ViAmpleHate. Bình luận được chuẩn hóa, tách từ và mã hóa bằng PhoBERT thành các hidden state $H = [h_0, \dots, h_n]$, trong đó h_0 là biểu diễn [CLS] toàn cục. NER tiếng Việt và target-cue bank tạo ra tập chỉ số target T_x , còn attack-cue bank tạo ra A_x (fallback về [CLS] khi một tập rỗng). Relation-bank attention gồm ba kênh theo từng mẫu—target tương minh (r_{exp}), context hàm ẩn (r_{imp}) và attack (r_{atk})—được fuse thành một relation vector duy nhất r rồi inject vào h_0 qua instance-adaptive gate ($g = \sigma(W_g[h_0; r])$, $z = h_0 + g \cdot r$). Linear classifier với threshold được tinh chỉnh trên tập validation cho ra quyết định NON-HATE/HATE; quá trình huấn luyện cực tiểu hóa $L = L_{\text{CE}} + \alpha L_{\text{CL}}$.

định về AmpleHate gốc; vì vậy, kết quả này được trình bày như ghi chú triển khai thay vì một thí nghiệm đánh giá riêng.

4.7 Instance-adaptive relation gate

Nhóm thay hệ số vô hướng cố định bằng một gate điều chỉnh theo từng mẫu:

$$g = \sigma(W_g[h_0; r] + b_g), \quad z = h_0 + g \cdot r. \quad (6)$$

Khi tín hiệu rõ ràng, mô hình tăng g để dựa nhiều hơn vào relation vector; khi cue yếu hoặc không có, mô hình giảm g và nghiêng về biểu diễn toàn cục. Vector z sau khi fuse đi qua dropout rồi một linear layer để tạo logit.

4.8 Hàm mục tiêu huấn luyện

Nhóm huấn luyện với $L = L_{\text{CE}} + \alpha L_{\text{CL}}$, $\alpha = 0.1$. Weighted cross-entropy $L_{\text{CE}} = -\sum_c w_c y_c \log \hat{y}_c$ xử lý mất cân bằng bằng cách tăng trọng số cho lớp HATE, kèm label smoothing (giá trị các class weight w_c , label smoothing và margin m được nêu trong Phụ lục A). Số hạng contrastive trên z (với $s_{ij} = \cos(z_i, z_j)$) kéo các cặp cùng lớp lại gần nhau và đẩy các cặp khác lớp ra xa:

$$L_{\text{CL}} = \frac{1}{N} \sum_{i \neq j} [\mathbb{I}[y_i=y_j](1 - s_{ij}) + \mathbb{I}[y_i \neq y_j] \max(0, s_{ij} - m)]. \quad (7)$$

4.9 Inference và chọn threshold

Khi inference, nhóm dùng một decision threshold được chọn trên tập validation theo hướng

cực đại hóa macro-F1, rồi cố định threshold này cho tập test:

$$t^* = \arg \max_t \text{MacroF1}(y, \mathbb{I}[p_{\text{HATE}} \geq t]). \quad (8)$$

4.10 Cân nhắc tính toán

So với PhoBERT, các thành phần thêm vào khá nhẹ: ba HeadAttention module và gate chỉ thao tác trên query [CLS] đã được pool cùng một số ít cue token của mỗi mẫu, nên chi phí phát sinh chủ yếu là vài linear projection; ngoài ra, batched attention đã chỉnh sửa còn rẻ hơn khi huấn luyện nhờ loại bỏ tương tác $B \times B$ giả tạo. Việc khớp cue cũng chỉ chạy một lần ở bước tiền xử lý.

5 Thực nghiệm

5.1 Các baseline

Nhóm so sánh với năm baseline, tất cả đều ở dạng nhị phân. **TF-IDF + LR** và **TF-IDF + SVM** dùng đặc trưng từ vừng thưa. **BiLSTM + fastText** (Bojanowski et al., 2017) kết hợp embedding tĩnh với một mô hình recurrent. **PhoBERT-CNN** xếp chồng CNN feature extractor lên PhoBERT. **AmpleHate-PhoBERT** là bản port của mô hình gốc với NER tiếng Anh, một HeadAttention đơn và injection cố định $z = h_0 + e r_{\text{base}}$ ($e = 1.0$), dùng chung encoder PhoBERT-base. Như trình bày trong Bảng 2, mỗi mô hình được chạy ở cấu hình dự kiến của chính nó: ViAmpleHate cố ý

dùng context window dài hơn (256) và effective batch lớn hơn, vì đây là một phần của hệ thống đề xuất. Do đó, phép so sánh ở đây là giữa toàn bộ hệ thống đề xuất và baseline ở cấu hình tương ứng, không nhằm cô lập đóng góp riêng của từng thành phần.

5.2 Chi tiết triển khai

Encoder là `vinai/phobert-base` (768-d) và NER tiếng Việt là `NlpHUST/ner-vietnamese-electra-base`. Max length được đặt ở 256; learning rate là $2e-5$ cho encoder và $5e-5$ cho head; dropout là 0.1; effective batch là 32 ($16 \times \text{grad-accum}$ 2). Nhóm huấn luyện tối đa tám epoch và giữ lại checkpoint tốt nhất theo macro-F1 trên tập validation; $\alpha = 0.1$ và threshold cũng được chọn trên validation theo macro-F1. Toàn bộ siêu tham số được liệt kê trong Phụ lục A.

5.3 Độ đo đánh giá

Hai độ đo chính là macro-F1 và HATE-F1, vì chúng phản ánh hiệu năng trên lớp thiểu số tốt hơn nhiều so với accuracy. Khi mẫu dương chỉ chiếm $\approx 10\%$, một bộ dự đoán luôn chọn lớp đa số đã đạt hơn 0,89 accuracy dù không phát hiện được trường hợp thù ghét nào. Macro-F1 lấy trung bình F1 trên các lớp, còn HATE-F1 tách riêng lớp mà nhóm quan tâm.

5.4 Thiết kế thực nghiệm

Ngoài điểm số cuối cùng, nhóm còn kiểm tra liệu mỗi thay đổi có khắc phục một hạn chế cụ thể của baseline hay không, từ độ phủ thấp, việc chưa mô hình hóa sự thù địch, đến cách hợp nhất tín hiệu và hàm mất mát. Riêng độ phủ target được khảo sát trực tiếp ở Mục 6.2.

Kết quả và phân tích lỗi

6.1 Kết quả chính

Mỗi mô hình được báo cáo ở đúng cấu hình dự kiến của nó (Mục 3.3, Bảng 2), nên các dòng đề xuất so với baseline phản ánh phép so sánh giữa toàn bộ hệ thống đề xuất và baseline ở cấu hình tương ứng. Mọi con số đều đến từ một lần chạy duy nhất (seed 42).

Các mô hình transformer vượt trội rõ rệt so với baseline từ vựng và embedding tĩnh ở những độ đo quan trọng (Bảng 3), cho thấy biểu diễn ngữ cảnh là cần thiết khi thù ghét phụ thuộc vào context, cách diễn đạt phi chính

thức và tương tác giữa lời nhắc tới target với vị từ thù địch. Trong nhóm transformer, baseline AmpleHate nhỉnh hơn một PhoBERT classifier thuần nhờ target-aware attention, nhưng lợi ích này bị giới hạn vì NER tiếng Anh hiếm khi tìm được target tiếng Việt hợp lệ, khiến hầu hết đầu vào quay về [CLS]. Nhóm chỉ so sánh với các mô hình tự huấn luyện theo cùng một giao thức; việc đối chiếu với kết quả ViHSD đã công bố được dành cho công trình tương lai.

Trên VOZ-HSD, ViAmpleHate cải thiện cả macro-F1 lẫn HATE-F1 với biên độ rõ ràng (HATE-F1 +0,0508). Trên ViHSD, khác biệt nhỏ hơn nhiều (+0,0027 macro-F1, +0,0036 HATE-F1). Vì mọi số liệu chỉ đến từ một lần chạy (seed 42) và chưa có kiểm định ý nghĩa thống kê, nhóm xem mức tăng trên ViHSD là kết quả sơ bộ. Đáng lưu ý, accuracy lại *giảm* trên VOZ-HSD trong khi macro-F1 và HATE-F1 đều tăng; đây là minh chứng cho thấy accuracy dễ gây hiểu nhầm khi dữ liệu mất cân bằng.

6.2 Tác động của trích xuất target tiếng Việt

Một yếu tố có thể góp phần là độ phủ target, tức tỷ lệ bình luận có $T_x \neq \{0\}$ (NER tiếng Việt hoặc một target cue được kích hoạt). Với NER tiếng Anh, chỉ $\approx 0,09\%$ bình luận train có target (21/24.048), nên nhánh target-aware của baseline gần như không hoạt động. Khi dùng NER tiếng Việt cùng target-cue bank, độ phủ target tăng lên 20,0%/18,8% trên 500 bình luận train/validation đầu của ViHSD và 45,2%/43,0% trên VOZ-HSD (cùng bank 16 cue); độ phủ attack cue thấp hơn, lần lượt là 5,2%/4,2% (ViHSD) và 7,6%/6,0% (VOZ). Đáng chú ý, bộ dữ liệu có độ phủ cao hơn (VOZ-HSD) cũng là nơi ViAmpleHate cải thiện nhiều nhất (HATE-F1 +0,0508 so với +0,0036), tạo ra một tương quan gợi mở. Tuy vậy, độ phủ chỉ là thống kê *đầu vào*: nó cung cấp thêm vị trí để relation-bank attention attend tới, nhưng chưa đủ để chứng minh rằng việc kích hoạt cue trực tiếp cải thiện từng quyết định.

6.3 Phân tích định tính

Để minh họa cụ thể các kiểu lỗi, Bảng 4 liệt kê một số dạng bình luận tiêu biểu. Đây là các ví dụ dựng lại theo những phạm trù quan sát được; trong phiên bản camera-ready, nên thay bằng mẫu test thực đã ẩn danh. Xu hướng

Thành phần	Baseline AmpleHate-PhoBERT	ViAmpleHate-PhoBERT (đề xuất)
Encoder	PhoBERT-base	PhoBERT-base
Trích xuất target	NER tiếng Anh	NER tiếng Việt + target-cue bank
Độ phủ target	$\approx 0,09\%$ tập train (21/24.048)	$\approx 18,8-20,0\%$ qua cue tiếng Việt
Tín hiệu attack	Không mô hình hóa	Attack-cue bank riêng
Attention	Một HeadAttention	Ba kênh: target / implicit / attack
Tính attention	matmul mức batch (trộn mẫu)	bm theo mẫu + masking
Fusion	Injection cố định $h_0 + er$ ($e=1.0$)	Relation bank + adaptive gate $h_0 + gr$
Loss	Weighted CE	Weighted CE + contrastive ($\alpha=0.1$)
Max length	128	256
Batch	16	$16 \times \text{grad-accum } 2$ (eff. 32)
NER khi đánh giá	Off ($\Rightarrow$ fallback [CLS])	On, nhất quán giữa các tập

Table 2: So sánh kiến trúc và cấu hình giữa baseline và mô hình đề xuất.

(a) ViHSD (test)

Mô hình	Acc.	Mac-F1	HATE-F1
TF-IDF + LR	0.8910	0.7393	0.5404
TF-IDF + SVM	0.9126	0.7131	0.4739
BiLSTM + fastText	0.8454	0.7072	0.5060
PhoBERT-CNN	0.8945	0.7571	0.5745
AmpleHate	0.9175	0.7792	0.6045
ViAmpleHate	0.9205	0.7819	0.6081
Δ vs base	$+0.0030$	$+0.0027$	$+0.0036$

(b) VOZ-HSD (test)

Mô hình	Acc.	Mac-F1	HATE-F1
TF-IDF + LR	0.9453	0.7745	0.5783
TF-IDF + SVM	0.9641	0.7831	0.5850
BiLSTM + fastText	0.8650	0.6712	0.4187
PhoBERT-CNN	0.9623	0.8150	0.6500
AmpleHate	0.9643	0.8185	0.6557
ViAmpleHate	0.9420	0.8371	0.7065
Δ vs base	-0.0223	$+0.0186$	$+0.0508$

Table 3: Kết quả trên tập test của ViHSD (a) và VOZ-HSD (b); AmpleHate là baseline, Δ là chênh lệch của ViAmpleHate so với baseline. Giá trị tốt nhất ở mỗi cột được in đậm (accuracy chỉ để tham khảo; riêng VOZ-HSD, baseline có accuracy cao nhất). Tất cả đều từ một lần chạy (seed 42).

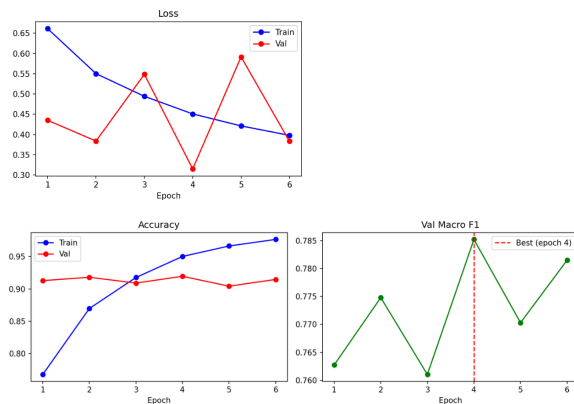


Figure 2: Đường cong training/validation của ViAmpleHate trên ViHSD (best epoch 4, macro-F1 trên validation = 0,7852). training_curves_viamplehate.png.

Kiểu bình luận	Tgt	Atk	Gold
Nhân nhóm + vị từ thù địch	yes	yes	HATE
Lời tục, không target nhóm	no	yes	NON
Mĩa mai/hàm ẩn, không cue lộ	no	no	HATE
Nhân nhóm trong context trung tính	yes	no	NON

Table 4: Các trường hợp minh họa (ví dụ dựng lại; nên thay bằng ví dụ thực). Tgt/Atk = có/hiện diện target/attack cue.

Dữ liệu	Lớp	P	R	F1
ViHSD	NON-HATE	0.9541	0.9574	0.9558
ViHSD	HATE	0.6177	0.5988	0.6081
VOZ	NON-HATE	0.9638	0.9719	0.9678
VOZ	HATE	0.7347	0.6803	0.7065

Table 5: Kết quả theo từng lớp của ViAmpleHate (tập test, một lần chạy).

chung là ViAmpleHate hỗ trợ tốt nhất khi một target tường minh đi kèm một attack predicate, và kém nhất với những trường hợp thù ghét hàm ẩn, tức không nêu target nào và cũng không có từ attack lộ liễu.

6.4 Thảo luận

Có ba điểm đáng chú ý. Thứ nhất, trên ViHSD, thay đổi chủ yếu là tái cân bằng precision/recall hơn là một mức tăng đồng đều: precision của HATE tăng ($0,5972 \rightarrow 0,6177$) trong khi recall *giảm* ($0,6119 \rightarrow 0,5988$). Precision cao hơn có lợi khi cáo buộc nhằm là tổn kém, nhưng recall thấp hơn đồng nghĩa với bỏ sót nhiều hơn; lựa chọn điểm cân bằng phù hợp phụ thuộc vào bối cảnh triển khai. Nhóm chỉ báo cáo ở một threshold đơn và để dành phân tích precision-recall đầy đủ cho sau. Trên VOZ-HSD, nơi độ

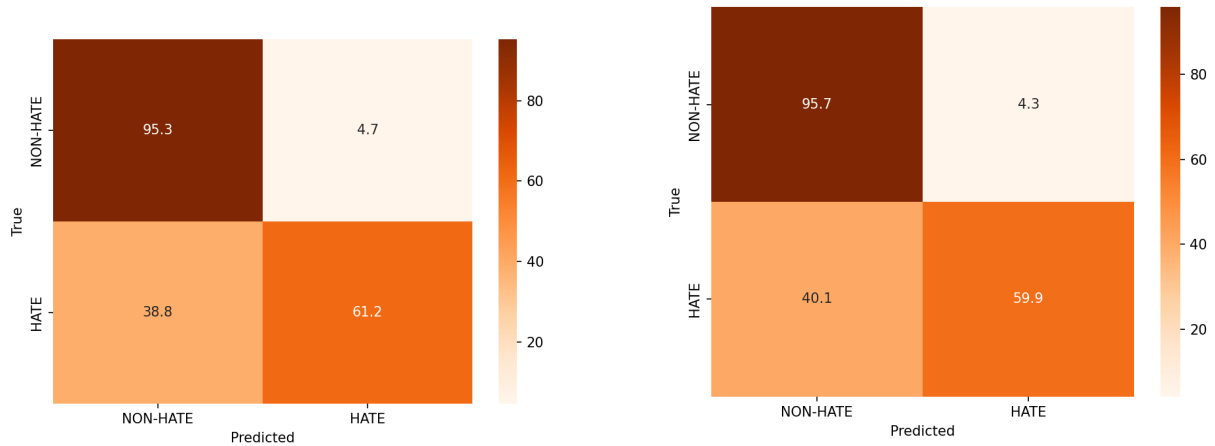


Figure 3: Confusion matrix trên ViHSD: AmpleHate baseline (trái) so với ViAmpleHate (phải). `confusion_matrix_amplehate.png` và `confusion_matrix_viamplehate.png`; lưu ý số false positive của lớp HATE giảm.

phủ cao hơn, recall của HATE cũng cao hơn (0,6803; Bảng 5). Thứ hai, mức tăng lớn hơn trên VOZ-HSD đi kèm accuracy giảm cho thấy mô hình đánh đổi accuracy của lớp đa số để cải thiện chất lượng ở lớp thiểu số, một đánh đổi thường phù hợp khi lớp thiểu số là trọng tâm. Thứ ba, quan hệ giữa độ phủ cue và mức cải thiện gợi mở một hướng nâng cấp ít tốn kém, đó là mở rộng và tinh chỉnh cue bank, dù vẫn cần kiểm chứng thêm (Mục 6.2).

6.5 Phân tích lỗi

Nhóm chia các lỗi còn lại thành bảy nhóm thường gặp. **(1) Xúc phạm nhưng không phải thù ghét:** lời lăng mạ hoặc lời tục nhắm vào một cá nhân, không phải một nhóm được bảo vệ; khi mô hình đặt trọng số quá lớn vào attack cue, false positive dễ xuất hiện. **(2) Thù ghét hàm ẩn:** bình luận không có slur, thực thể có tên hay attack predicate lộ liễu, nhưng vẫn thể hiện thù địch qua mỉa mai, định kiến hoặc ngữ cảnh xã hội chung; lúc này việc trích cue gần như không tìm được tín hiệu để attend. **(3) Tham chiếu target mơ hồ:** đại từ và danh từ chỉ nhóm cũng xuất hiện trong bình luận trung tính hoặc hài hước, nên một target cue thiếu ngữ cảnh thù địch dễ khiến mô hình dự đoán nhầm thành HATE. **(4) Cue chưa phủ đủ:** tiếng lóng thay đổi nhanh, biến thể chính tả, viết tắt, lời tục sáng tạo hoặc bị kiểm duyệt là những hiện tượng mà cue bank cố định khó bao quát hết, dẫn tới false negative. **(5) Lệnh token và span:** span của NER, kết quả tách từ và subword tokenization của

PhoBERT có thể không trùng khớp, nhất là với các cụm nhiều từ, khiến cue đôi khi khớp sai vị trí. **(6) Nhạy cảm với threshold:** threshold tinh chỉnh trên validation có thể không còn phù hợp khi phân phối dữ liệu thay đổi. **(7) Mất cân bằng lớp:** khi số mẫu dương quá ít, lỗi ở lớp thiểu số vẫn dai dẳng dù đã dùng weighted loss và tinh chỉnh threshold. Các hướng khắc phục gồm mở rộng cue bank bằng khai thác dữ liệu có kiểm chứng thủ công, ghi log dự đoán theo từng mẫu (gate, độ phủ cue, confidence) để phân tích false positive/negative, bổ sung span supervision hoặc phát hiện mỉa mai, và áp dụng probability calibration.

7 Kết luận và hướng phát triển

ViAmpleHate thích ứng AmpleHate cho tiếng Việt thông qua trích xuất target tiếng Việt, tách riêng hai kênh target/attack cue, relation-bank attention và relation injection thích ứng (kèm một chỉnh sửa batched-attention ở mức triển khai), đồng thời huấn luyện bằng weighted cross-entropy kết hợp contrastive loss. Trên ViHSD và VOZ-HSD, mô hình cải thiện cả macro-F1 lẫn HATE-F1 so với baseline AmpleHate và các baseline TF-IDF, BiLSTM, PhoBERT-CNN, rõ nhất ở lớp thiểu số của VOZ-HSD; mức tăng trên ViHSD còn nhỏ và cần xác nhận lại bằng so sánh đa seed. Phân tích cho thấy các mức cải thiện gắn với độ phủ target, vốn được cue bank tiếng Việt nâng lên khoảng hai bậc độ lớn so với NER tiếng Anh; tuy nhiên, đây mới là liên hệ tương quan và chưa thể khẳng định nhân quả. Bài học

rộng hơn là target-aware chỉ phát huy đầy đủ khi được thích ứng với hình thái bề mặt của ngôn ngữ đích. Hướng phát triển tiếp theo gồm tự động khai thác cue bank kèm kiểm chứng thủ công, phân tích lỗi ở mức từng mẫu, bổ sung span supervision và phát hiện mĩa mai, mở rộng sang thiết lập multi-label/multi-target theo tinh thần ViTHSD, và áp dụng probability calibration để ổn định hơn khi chuyển miền.

References

- Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146.
- Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. 2020. Supervised contrastive learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Yejin Lee, Joonghyuk Hahn, Hyeseon Ahn, and Yo-Sub Han. 2025. [AmpleHate: Amplifying the attention for versatile implicit hate detection](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 28862–28874, Suzhou, China. Association for Computational Linguistics.
- Son T. Luu, Kiet Van Nguyen, and Ngan Luu-Thuy Nguyen. 2021. [A large-scale dataset for hate speech detection on vietnamese social media texts](#). In *Advances and Trends in Artificial Intelligence. Artificial Intelligence Practices*, volume 12798 of *Lecture Notes in Computer Science*, pages 415–426, Cham. Springer.
- Dat Quoc Nguyen and Anh Tuan Nguyen. 2020. PhoBERT: Pre-trained language models for Vietnamese. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1037–1042.
- Luan Thanh Nguyen. 2024a. [ViHateT5: Enhancing hate speech detection in vietnamese with a unified text-to-text transformer model](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 5948–5961, Bangkok, Thailand. Association for Computational Linguistics.
- Luan Thanh Nguyen. 2024b. [VOZ-HSD: A hate speech detection dataset from the voz forum](#). Hugging Face dataset. Dataset released with the paper ViHateT5: Enhancing Hate Speech Detection in Vietnamese with a Unified Text-to-Text Transformer Model.
- Cuong Nhat Vo, Khanh Bao Huynh, Son T. Luu, and Trong-Hop Do. 2025. [ViTHSD: Exploiting](#)

[hate by targets for hate speech detection on vietnamese social media texts](#). *Journal of Computational Social Science*, 8(2):30.

A Hyperparameters

Tham số	Giá trị
Encoder	vinai/phobert-base (768-d)
NER tiếng Việt	NlpHUST/ner-vietnamese-electra-base
max_len	256
LR (encoder / head)	2e-5 / 5e-5
Dropout	0.1
Effective batch	32 (16 × grad-accum 2)
Epoch	tối đa 8 (chọn tốt nhất theo macro-F1 trên val)
α (contrastive)	0.1
Class weight w_c	inverse freq. $N/(C n_c)$
Contrastive margin m	0.5
Label smoothing	0.05
Seed / số lần chạy	42 / một lần chạy
ViHSD best epoch / val F1 / thr.	4 / 0.7852 / 0.43

Table 6: Toàn bộ siêu tham số.

B Ví dụ cue bank

Hai bank đầy đủ—16 target cue và 24 attack cue—được phát hành kèm mã nguồn và liệt kê trong báo cáo đi kèm; tại đây, nhóm chỉ nêu một vài ví dụ an toàn ASCII. **Target cue** mang tính tham chiếu và bản thân không hàm ý thù ghét: các cách gọi người/nhóm phi chính thức, nhãn vùng miền và nhân khẩu, đại từ nhóm. **Attack cue** là các vị từ thù địch thể hiện sự khinh miệt, phi nhân tính hóa hoặc ăn bám (ví dụ [khinh](#), [an_bam](#)). Hai loại cue này được lưu trong các bank tách biệt (Mục 3.4).

C Khả năng tái lập

Mọi mô hình dựa trên PhoBERT đều dùng chung [vinai/phobert-base](#); điểm khác nhau giữa baseline và ViAmpleHate nằm ở cách mô hình hóa target/relation, phép tính attention, fusion và loss. Checkpoint tốt nhất được chọn theo macro-F1 trên validation, còn threshold cho lớp HATE được chọn trên validation rồi cố định cho test. Việc khớp cue chỉ chạy một lần ở bước tiền xử lý, theo kiểu khớp trọn chuỗi token với dòng token của PhoBERT.